

Analysis and Feedback

Executive assessment

This assignment is primarily a concurrency and data-integrity exercise presented as a small full-stack feature. The strongest submission will be narrow, runnable, and explicit about invariants. A visually ambitious frontend would add little value if the final-seat race or payment-state transitions are weak.

The brief is well scoped for a 3-4 hour timebox, but it intentionally leaves several choices open. Those choices should be documented as assumptions rather than hidden in code.

What the reviewers are likely testing

- Whether correctness is enforced in the backend and database rather than trusted to the UI.
- Whether the last-seat race is demonstrated by a deterministic concurrent test, not only described.
- Whether payment failure and seat allocation are separate state transitions.
- Whether duplicate and replayed requests are safe.
- Whether the candidate can cut scope and still deliver a coherent end-to-end path.
- Whether design tradeoffs are explained in plain language.

Ambiguities to resolve explicitly

Does selecting a class reserve a seat?

Recommended assumption: no. Selection and creation of a `pending_payment` booking do not reduce availability. Only `confirmed` bookings occupy seats. This matches the required race scenario because both users can reach payment for the last seat.

What happens if payment succeeds after the final seat is gone?

Recommended take-home behavior: record the successful payment attempt, set the booking to `capacity_unavailable`, do not add it to the roster, and return a response explaining that a refund/void would be required in production.

Recommended production behavior: authorize the payment, atomically claim the seat, then capture. If the seat claim fails, void the authorization. Do not pretend an external payment provider can participate in the database transaction.

Can the same child make multiple pending bookings for one class?

Recommended assumption: no. Enforce one active booking (`pending_payment` or `confirmed`) per student and class. Allow a new attempt after `payment_failed`, `capacity_unavailable`, or `cancelled`.

Are parents, teachers, and admins authenticated?

Recommended assumption: authentication and authorization are outside the timebox. Seed a parent and expose a clearly labelled demo admin roster route. State that production endpoints require ownership and role checks.

Are cancellations and expired pending bookings required?

No. They can exist in the state model to show extensibility, but only implement them if the core flow and tests are complete.

Suggestions that improve the submission

1. Use a compact spec-driven sequence: requirements and acceptance criteria, technical plan, ordered tasks, implementation, then verification.
2. Put invariants near the top of the README. Reviewers should understand correctness before reading setup details.
3. Include one deterministic concurrency test that starts two confirmations together against a class with three confirmed students, then asserts exactly one success and a final roster size of four.
4. Make the roster query filter by `confirmed` in the data access layer rather than filtering in the UI.
5. Use an idempotency key for booking creation and payment-result recording. Payment providers retry callbacks; a senior-level solution should acknowledge this.
6. Keep payment simulation intentionally simple: a success/failure selector or endpoint is enough.
7. Document why a database transaction is used now, why a Saga is deferred, and what would justify Kafka later.
8. Provide a seed command and a single command that runs all tests.
9. In the walkthrough, show the failure paths before discussing frontend polish.
10. Track the actual time spent and stop when the four-hour cap is reached. Notes about unfinished polish demonstrate scope control.

Common weak approaches to avoid

- Checking `availableSeats > 0` in the browser and then inserting without a database transaction.
- Counting seats and inserting in separate, unlocked database operations.
- Treating a unique constraint as sufficient for capacity; uniqueness prevents duplicates but not a fifth distinct student.
- Holding a database transaction open while waiting for a network payment call.
- Adding a child to the roster while payment is pending.
- Relying only on manual testing for the race condition.
- Using an in-memory store while claiming that concurrent correctness is solved for multiple server instances.
- Adding a distributed Saga when the implemented flow has one transactional database boundary.
- Adding Kafka for a single producer and consumer without replay, scale, or independent-service requirements.
- Using full GitFlow for a short exercise with no release or hotfix lifecycle.
- Expanding into enrollment, scheduling, notifications, or a design system.

Scope recommendation

Must have

- Seeded parent, children, and classes.
- Class list with computed availability.
- Booking creation.
- Mock payment success/failure.
- Booking status display.
- Confirmed-only roster.
- Duplicate protection.
- Atomic last-seat handling.
- Automated happy-path, duplicate, failure, and race tests.
- README and AI_USAGE documentation.

Nice to have

- Minimal browser UI.
- OpenAPI document.
- Structured logs and health endpoint.
- Pending booking expiry.
- A concise production-evolution note covering authorization/capture, compensation, an outbox, and the conditions that would justify Kafka.

Deliberately exclude

- Regular enrollment.
- Real payment-provider integration.
- Full authentication/authorization.
- Email/SMS notifications.
- Complex admin tools.
- Production deployment infrastructure.